Travel that keeps its *context.*

A business-travel workspace that turns a confirmed ticket into sourced advice, a grounded assistant, and an invoice that does not need chasing.

BUILT ON
Vultr · Docker · Supabase

LIVE WEB
Bright Data

DEPLOYMENT
travis.nodesemesta.com

CHALLENGE
lablab.ai · Powering the Future of Work

MUHAMAD JAMALUDIN

01 BACKGROUND

The booking is the beginning, not the product.

Business travel is sold as a sequence of reservations. The real work starts after the last confirmation email, when the traveler has to turn a receipt into a day.

Modern travel platforms are excellent at discovery: find a flight, compare a hotel, hold a rate, pay. Once the booking is confirmed, the experience fragments. The confirmation lives in an inbox, the meeting in a calendar, the venue in a map, the weather in another tab, the invoice in a finance portal. The traveler becomes the integration layer, repeating the same context to every tool.

That repetition is not just friction. It is where errors enter the trip. A hotel address copied into a route request, a meeting time remembered in the wrong timezone, a local event missed because nobody thought to search for it, a receipt chased a week after the journey. The traveler does not need another search box; they need the trip to remain one object.

Large language models make a new interface possible. A traveler can ask a question in their own words and get an answer that combines schedule, place, weather and movement. But a generic assistant is stateless. It does not know which trip is open, which ticket has been paid, which place is resolved, or which answer came from a live page versus a plausible guess. In travel, plausible is not good enough.

Travis was built for the lablab.ai Challenge 2, “Powering the Future of Work”, around a narrower and more useful idea: the trip is the context, and every answer must be traceable to it. The first tested scenario is a business trip to Jakarta, with a hotel, a convention centre and an airport. The build is deployed on Vultr and connected to Bright Data for live web data.

THE SHIFT TRAVIS MAKES

1 trip object holding places, agenda, legs and bookings

9 read-only assistant tools over that context

4 live data providers behind one backend

29 database tables with row-level security

WHAT CHANGES AFTER BOOKING

- Nearby events become part of the trip, not a separate search.
- The assistant knows the place and the date before it answers.
- Every live fact carries a source and a retrieval time.
- Invoices are generated from the same record as the booking.

THE OBJECT BEHIND THE VIEWS

Place · Agenda · Journey leg · Timezone · Booking · Invoice

> “The gap is not another search box. It is continuity: one trip that remembers what it already knows.”

WITHOUT A TRIP OBJECT

The traveler re-explains the journey to every tool. Context lives in tabs, memory and screenshots, and each answer starts from zero.

WITH TRAVIS

Place, agenda, legs, timezone, booking and invoice stay attached to one trip. Every view reuses the same record instead of asking again.

Scope of this paper: the deployed product, its architecture and the decisions behind it. The tested scenario is one business trip in Jakarta; multi-city and production fulfillment are treated as future work.

02 PROBLEM

Four failures that show up after the confirmation.

The product problem is not a missing feature in a booking flow. It is the absence of a durable, trustworthy operating context for the days around the booking.

01

Context fracture

Confirmation, calendar, venue, route, weather and invoice live in different systems. Each one asks the traveler to re-enter what the previous one already knew. The trip exists in the traveler's head, not in the software.

02

Repeated questions

“What is near the venue?”, “How long is the ride?”, “Will it rain?”, “Where is the invoice?” These are not four products. They are four views of one trip, asked in the order the day demands.

03

Unverifiable answers

A general assistant can produce a confident answer with no relationship to the trip, the date or a source. A business traveler cannot act on a venue that does not exist or a route that was never computed.

04

Operational fragility

Live providers fail, rate-limit, return captchas and change their answers. A product that silently substitutes stale data or a fixture is worse than one that says “this source is unavailable”.

Existing tools solve slices. Calendar apps know the meeting but not the ride. Maps know the route but not the ticket. Booking platforms know the reservation but stop at the confirmation screen. An LLM chat can synthesize, but only if it is given the right context and constrained by the right evidence. Travis therefore treats context, provenance and failure as first-class product requirements, not implementation details.

Read-only by default
Answers cannot silently change the itinerary.

Sources or nothing
Live facts carry a URL, provider and retrieval time.

No silent fallback
Provider failure is a structured error, not invented data.

Confirmation for change
An itinerary mutation needs an explicit token and user action.

TRAVELER
Carries the trip in their head, re-asks every tool, and discovers a missed event too late.

OPERATIONS
Cannot see whether advice was sourced, stale or never fetched; support starts with an interview.

FINANCE
Reconciles a payment with a booking and an invoice that were never designed to meet.

THE SYMPTOM

Every new question starts with a new search. The answer may be right, but it is not attached to the trip, the date or a source, so it cannot be acted on with confidence.

THE DESIGN RESPONSE

One trip record, one backend, one evidence trail. The assistant and the Advice view read the same context, and a provider failure is surfaced instead of hidden.

> The assistant must know the trip before it can answer; it must show the source before it can be trusted; it must fail honestly when a provider fails.

03 SOLUTION

One trip, three ways to use it.

Travis turns the confirmed trip into a workspace. The same context powers a structured advice view, a conversational assistant and the operational records behind the journey.

01

The trip is the object

Places, agenda items, journey legs, timezone, bookings and invoices live under one trip. The workspace opens on that trip, not on a generic chat.

Trips · Itinerary · Alerts · Invoices

02

Advice arrives without a prompt

When the trip has a ticket, the Advice view reads its place and date, then asks Bright Data for local events, OpenWeather for the forecast and the mobility engine for the ride. No question is typed.

Bright Data · OpenWeather · HERE

03

The assistant stays in the margin

A collapsible right-hand chat answers from the same trip context. It plans with an allowlist of tools, validates every number and source, and asks for confirmation before changing the itinerary.

Vultr Serverless Inference · 9 read tools

A day in the product. A traveler books a stay in Jakarta. The booking creates a place, a time and an agenda item on the trip. When the workspace opens, Explore shows the next decision, Advice already has the local picture, and the assistant is available without taking over the screen. The traveler can ask “what should I know about today?”, open the event that matters, compare the ride, and find the invoice after payment.

BOOK	CONTEXT	ADVICE	ASK	CLOSE
Stay or flight creates the ticket and its place.	Trip stores agenda, legs, timezone and payment.	Events, weather and route are fetched and sourced.	The assistant answers from the trip, with citations.	Invoice PDF and delivery follow the payment.

Explore
Search stays and flights with the trip already in view.

Advice
Automatic, sourced local picture for the next ticket.

Itinerary
The day as a timeline, with recommendations and actions.

Alerts
Departure, traffic and weather risks that changed.

Invoices
The paid record, PDF and delivery status.

Trips
The source of truth: places, tickets and payments.

> The product is not a chatbot with a travel theme. It is a trip that can explain itself.

Advice is generated when the Advice view opens for a trip that has tickets. Moving the trigger to the booking event is future work, described on page 07.

A small stack with explicit boundaries.

Every component has one job. The browser never calls a provider; the model never touches the database; the worker never changes the itinerary.

LAYER	CHOICE	WHY IT IS THERE
Web	Next.js 15 App Router, React 19, TypeScript	Authenticated workspace, server-rendered shell, view-level code splitting.
API	Node.js 22 + TypeScript, no framework	Explicit routing, token verification, ownership checks and provider orchestration.
Domain	Pure TypeScript packages	Trip rules, mobility engine `mobility-v1`, alert rules, invoice rules — deterministic and testable.
Data	Supabase Postgres, Auth, Storage; RLS on all 29 tables	Ownership is enforced in code and again in the database. Invoices live in a private bucket.
Infrastructure	Vultr VM, Docker Compose, Caddy, self-hosted Supabase	One deployable unit: web, API, worker, database and HTTPS on a single server.
AI	Vultr Serverless Inference, `deepseek-v4.1-flash-1017`	Planner tool calls, answer synthesis and evidence-backed event claim extraction.
Live web	Bright Data Direct API: SERP + Web Unlocker	Search results and page contents, server-side only. Bing SERP is used because the Google zone returns captcha headers.
Travel data	HERE routes, transit, traffic; OpenWeather	Deterministic travel time, departure advice, forecast and risk signals.
Commerce	LiteAPI sandbox; internal demo pricing	Hotel and flight search, prebook, sandbox payment and reconciliation. No real card is charged.
Contracts	OpenAPI, 27 JSON Schemas, no regular expressions	The model output, API bodies and providers are validated against shared shapes.

Server-side only · Bright Data · HERE · OpenWeather · Vultr Inference · LiteAPI

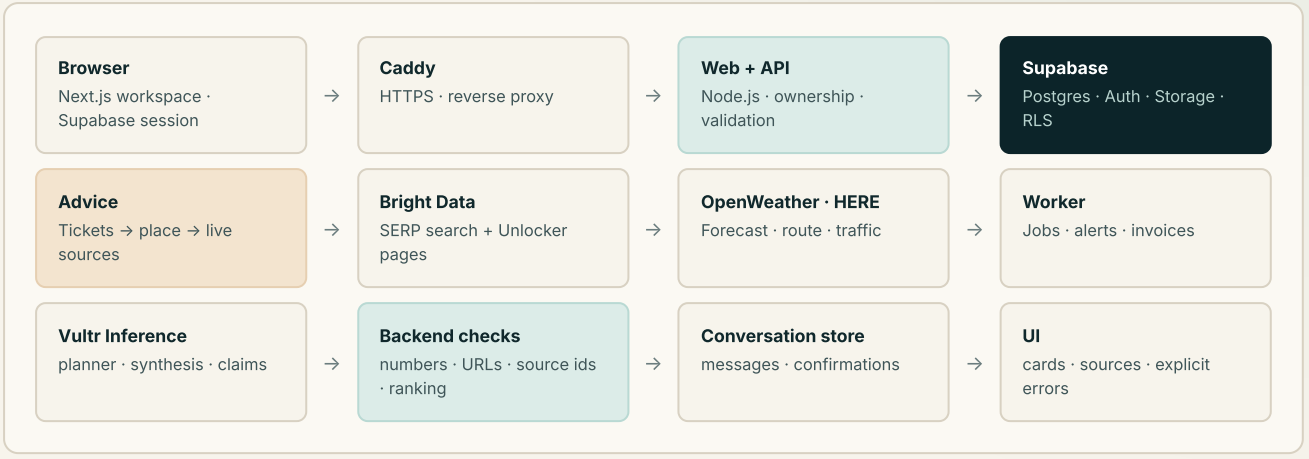
Vultr in the critical path. The same VM runs the web app, the API, the worker and the self-hosted Supabase stack behind Caddy. Vultr Serverless Inference is called for the planner and synthesis on every grounded answer; the model id is echoed back and checked, because an unknown id is silently rerouted by the API.

Honest boundaries. Hotel and flight fulfillment is sandbox. Payment is simulated. Email depends on the deployment's SMTP configuration. The whitepaper reports the deployed state, not a future production contract.

05 ORCHESTRATION

Three paths, one source of truth.

The orchestration is intentionally boring: the backend decides, providers answer, the model writes, and validation decides what reaches the traveler.



Advice

Tickets define the place and window. Weather, events and the nearest ride are fetched in parallel. Each result keeps its source and `retrieved_at`; the view renders an empty state rather than a guess.

Assistant

The backend resolves the place. A forced tool call selects from an allowlist. Tools run server-side. Synthesis returns strict JSON; the backend rejects invented numbers, unknown URLs, missing sources and any change to the computed ranking.

Booking → invoice

A server quote is paid through a Postgres RPC with a row lock. A succeeded payment enqueues invoice generation; the worker writes the PDF, stores it privately and sends it after a checksum match.

Model has no DB or provider access

Web content is fenced as untrusted

Provider calls are never retried

Confirmation token is hashed and single-use

Failures are structured, never fabricated

1

Ticket and place are read from the trip.

2

Weather, events and ride are requested in parallel.

3

Results are validated, timestamped and stored.

4

The view renders sourced cards, or an explicit empty state.

5

A failure is shown as a failure; nothing is cached as truth.

Provider rejects

Bright Data captcha, rate limit or unreadable page → structured provider code.

API answers

502 or 429 with request id and a safe message. No retry, no fixture.

UI responds

The card states what failed and offers a refresh; an empty result stays empty.

The orchestrator is also the trust boundary. The browser holds only the user token. The service-role key stays in the API and worker. Provider keys remain server-side, and logs redact secret-like fields.

06 FUTURE WORK

From a verified demo to a travel pilot.

The hard part is already visible: keep the trip coherent while more providers, more places and more people enter it. The next work is about coverage and operational confidence, not a new interface.

NEXT

Make advice event-driven

- Trigger the first scrape from the booking worker, not only when Advice opens.
- Refresh advice when a ticket is added, changed or cancelled.
- Add a second SERP engine behind the same adapter; Bing is a fix, not a strategy.
- Per-route rate limits for event discovery and chat.

NEXT 90 DAYS

Close the production gaps

- Production payment and provider fulfillment behind the sandbox boundary.
- Streaming answers only after validation, not before.
- Push and email alerts; event-change and transit-disruption alerts.
- Geocoder in the place form; onboarding and profile.
- Multi-city and multi-trip context.

LATER

Operate a travel product

- Backups, restore drills and retention policy.
- Dashboards, alerting, log shipping and provider SLOs.
- Team policies, expense reconciliation and approval flows.
- Offline itinerary and a mobile-first companion.

What we will not do. Fill a provider outage with stale or invented content. Let the model set a price, choose a route or change an itinerary. Treat a fixture as a live result. Those constraints are the product, not a temporary limitation.

Pilot shape

One travel team, one city and one provider per category. Advice is enabled for a small set of trips; every answer is reviewed for source quality before the scope widens.

Exit criteria

Advice opens with no prompt, every live fact has a source and retrieval time, and a provider failure is visible without blocking the rest of the trip.

Coverage
Places and dates resolve for every ticket in the pilot.

Trust
No unsourced number or venue appears in a card or answer.

Recovery
A failed job or provider is visible and rerunnable.

PROVEN IN THE BUILD

Advice from a ticket, a grounded assistant, sandbox booking, payment, invoice, worker jobs and a public HTTPS deployment.

NEEDS A PILOT

Provider quality across cities, production fulfillment, alert delivery, operational runbooks and a real support loop.

The roadmap follows the evidence. Advice already works from the ticket, the assistant already answers from tools, and the invoice already follows payment. The next milestone is not “more AI”; it is making each of those paths automatic, observable and recoverable when a provider or a worker fails.

A production pilot should start with one team, one city and a small set of providers — the same shape as the current demo — then widen geography only after the alert and reconciliation paths are proven.

07 CONCLUSION

Context is the product.

Travis is a small answer to a large gap: after the booking, the traveler should not have to become the integration layer for their own trip.

01

The trip is the interface

A durable object — places, agenda, legs, timezone, bookings — gives every answer a home and every view the same truth.

02

Evidence is a feature

Live facts carry a provider, a URL and a retrieval time. The assistant is allowed to say “unavailable”; it is not allowed to guess.

03

Honest failure earns trust

No retries, no silent fallback, no fixture passed off as live. Structured errors tell the traveler what happened and what to do next.

Built on Vultr, Travis runs as one deployable unit: a Next.js workspace, a Node.js API, a background worker and a self-hosted Supabase stack behind Caddy. Vultr Serverless Inference handles the language work; Bright Data supplies the live web; HERE and OpenWeather supply movement and weather. The model never becomes the source of truth. The backend validates every number, URL, source id and ranking before a traveler sees it.

The result is deliberately narrower than a general travel agent, and more useful because of it. Advice appears from a ticket without a prompt. The assistant answers from the trip with citations. A booking can become a paid record, an invoice and an email. The demo is live at `travis.nodesemesta.com`; the next step is a pilot with a real travel team and a production provider boundary.

> Travel should not require the traveler to remember what the software forgot.
>
> TRAVIS · TRAVIS.NODESEMESTA.COM · BUILT ON VULTR WITH BRIGHT DATA

LIVE TODAY

A deployed workspace, automatic advice from tickets, a grounded assistant, sandbox booking, payment, invoice and a background worker.

NEXT

Event-driven advice, production fulfillment, alert delivery, multi-city context and the operational controls a pilot needs.

29 tables	45 paths	27 schemas	9 tools
Postgres with RLS on every table.	Documented in the OpenAPI contract.	Shared JSON validation for answers.	Read-only assistant actions on the trip.

The build is intentionally small enough to inspect: one VM, one Compose file, one contract. That is what makes the next step a pilot rather than a rewrite.

This paper describes the deployed build as of November 2026. The tested scenario is a single business trip in Jakarta; sandbox commerce and simulated payment are labelled as such. Claims about live providers reflect the deployment at the time of writing.